Optimization for Deep Learning (OPT4DL)

Abolfazl Hashemi, Purdue ECE

Stochastic Gradient Descent (SGD): Some Examples

Linear Regression

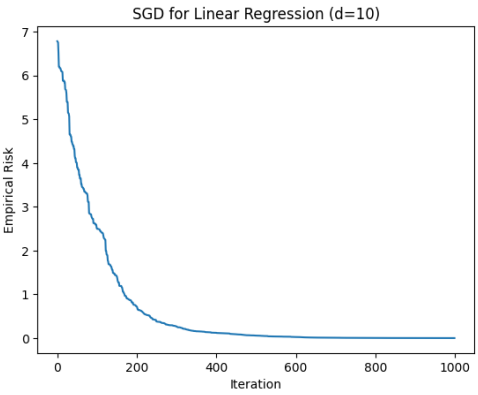
- Linear Regression
  - Features: $x_i \in \mathbb{R}^d$, $i = 1, \ldots, N$
  - Labels: $y_i \in \mathbb{R}$, $i = 1, \ldots, N$
  - Linear dependency: $y = x^\top w + e$ where $e \sim \mathcal{N}(0, \sigma^2)$
  - Square loss: $\ell(w, z) = \frac{1}{2}(y - x^\top w)^2$

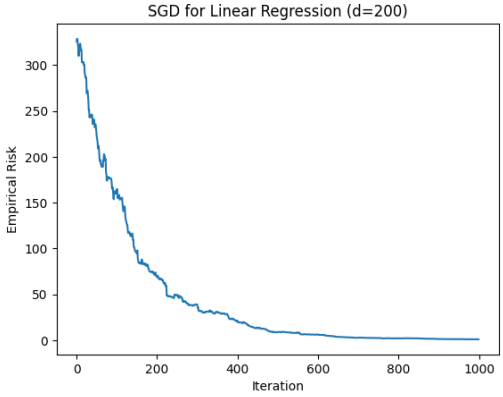
```
## Data Generation
X = np.random.normal(0, 1, (N,d))
w_best = np.random.normal(0, 1, (d, 1))
e = np.random.normal(0, sigma, (N, 1))
y = X @ w_best + e # True model
# Random initialization of weights
w = np.random.normal(0, 1, (d, 1))
# History for loss
loss1 = np.zeros(T)

for i in range(T):
    loss1[i] = np.mean(np.power(y-X@w, 2)) # save loss
    idx = np.random.randint(0, N) # choose a sample randomly {uniform dist.}
    xi, yi = np.expand_dims(X[idx], axis=1), y[idx]
    gw = -2*(yi-w.T @ xi)*xi # calculate gradient
    w -= eta * gw # update the weights
```

Linear Regression (Cont'd)

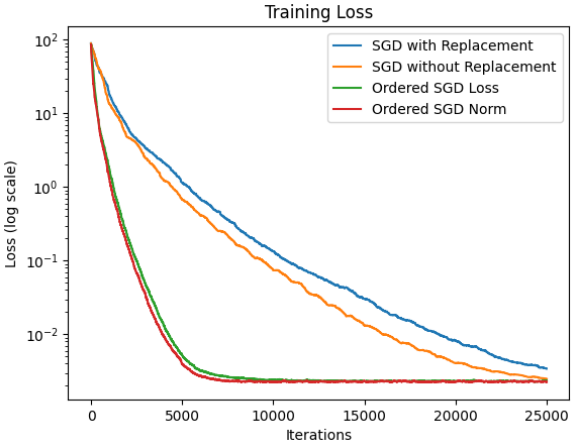
- $N = 100,\ d = 10,\ 200$





Non-Linear Regression

- $\ell(w, z) = \frac{1}{2}(y - \sigma(x^\top w))^2$
- A simple neural network



Performance-Robustness Tradeoff

- Focusing on harder instances accelerates training by finding the boundary sooner
- Typically measured by loss values $\ell(w_t, z_t^i)$ or gradient values $\|\nabla \ell(w_t, z_t^i)\|$
- Performance-Robustness Tradeoff: More chance that samples near the boundary have noisy labels, thus focus on easy samples for robust training
- Many such tradeoffs between performance and other contexts in ML/DL optimization

